

Name: Mandeep Singh

Roll No: 2301720049

Program: BTech CSE (Data Science)

DATE .....

COA

(Computer Organization & Architecture)

Assignment ~ 3

(Section ~ A)

### (1) Clocking Methodology

- Clocking methodology refers to the technique used to synchronize all operations inside a processor using a clock signal.
- The clock generates periodic pulses, & all components like registers, ALU & memory perform their operations in coordination with these pulses (usually on rising or falling edges).
- This ensures data is transferred in a controlled & predictable manner, avoiding timing errors & ensuring controlled execution of instructions.

### (2) Clock Cycle:

- Clock cycle is the smallest unit of time in a processor, defined as the time between two consecutive clock pulses.
- Each operation inside the CPU (like fetching, decoding, executing) takes one or more clock cycles.
- The duration of a clock cycle determines the processor speed - shorter cycles mean higher frequency & faster execution.

## (3) Amdahl's Law

- Amdahl's law describes the theoretical limit of performance improvement when only a part of a system is improved (e.g. parallelized). It states that the speedup of a program depends on the fraction that can be ~~part~~ parallelized.

- It is given by:

$$\text{Speed up} = \frac{1}{(1-p) + \frac{p}{S}}$$

Where:  $P$  = portion of program that can be parallelized  
 $S$  = speedup of that portion

- Even if the ~~part~~ parallel portion is very fast, the non-parallel part limits the total performance.

## (4) Data Path

- The data path is a collection of hardware components responsible for processing data within the CPU.
- It includes registers (for storing data), ALU (for computations), buses (for data transfer) & multiplexers (for selecting inputs)
- It defines how data flows & gets transformed during instruction execution.

## (5) Control Unit:

- The control unit acts as the "brain" of the processor. It generates control signals based on the instruction being executed.
- These signals determine:
  - (i) which registers to read/write
  - (ii) what operation the ALU should perform
  - (iii) whether to access memory
- It ensures proper ~~coordination~~ coordination b/w all components in the datapath.

## (6) Single-Cycle Processor:

- In single-cycle processor, every instruction (fetch, decode, execute, memory access, write-back) is completed within one ~~block~~ single clock cycle.
- Although simple in design, it's inefficient because the clock must be long enough to accommodate the slowest instruction, leading to wasted time for simpler instructions.

## (7) Multi-Cycle Processor:

- A multi-cycle processor divides instruction execution into multiple stages, each performed in a separate clock cycle.
- This allows:
  - (i) shorter clock cycles
  - (ii) better reuse of hardware
  - (iii) improved efficiency

## (8) Instruction Pipelining:

- Instruction pipelining is a technique where multiple instructions are executed in overlapping fashion by dividing execution into stages (IF, ID, EX, MEM, WB).
- Each stage ~~processor~~ processes a different instruction simultaneously, increasing throughput (instructions/unit time) without increasing clock speed.

## (9) Instruction-Level Parallelism (ILP)

- ILP refers to the ability of a processor to execute multiple independent instructions at the same time.
- It is achieved using techniques like:
  - (i) Pipelining
  - (ii) Superscalar execution
  - (iii) Out-of-order execution
- High ILP leads to better CPU performance.

## (10) Data Hazards

- Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed.
- Types include:
  - RAW (Read After Write)
  - WAR (Write After Read)
  - WAW (Write After Write)
- These ~~hazards~~ hazards can cause incorrect ~~results~~ if not handled properly.

DATE .....

## (11) Control Hazards

- Control Hazards arise due to branch or jump instructions that change the program flow
- Since the next instruction is uncertain, the pipeline may fetch incorrect instructions, causing delays or flushing.

## (12) Hazard Mitigation

- Pipeline hazards can be reduced using:
    - Forwarding: Passing results directly b/w pipeline stages
    - Stalls: pausing pipeline execution
    - Branch Prediction: Guessing branch outcomes
    - Instruction Reordering: Rearranging instructions to avoid conflicts
- These techniques improve pipeline efficiency.

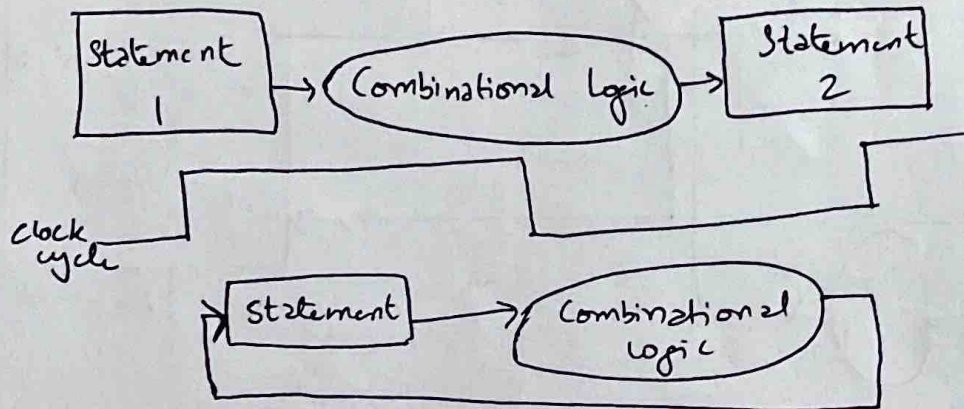
## (Section - B)

### \* Diagram - Based

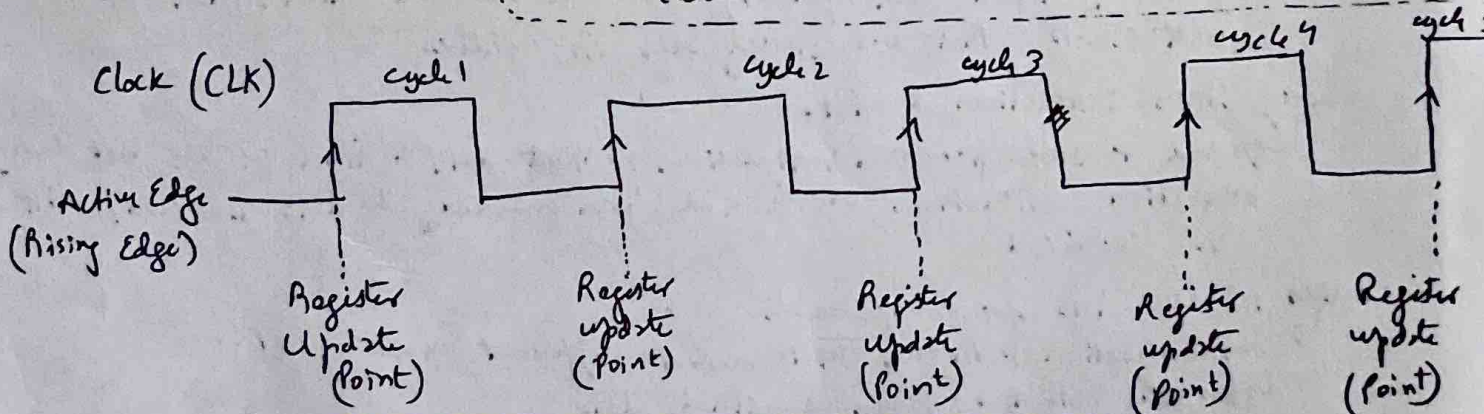
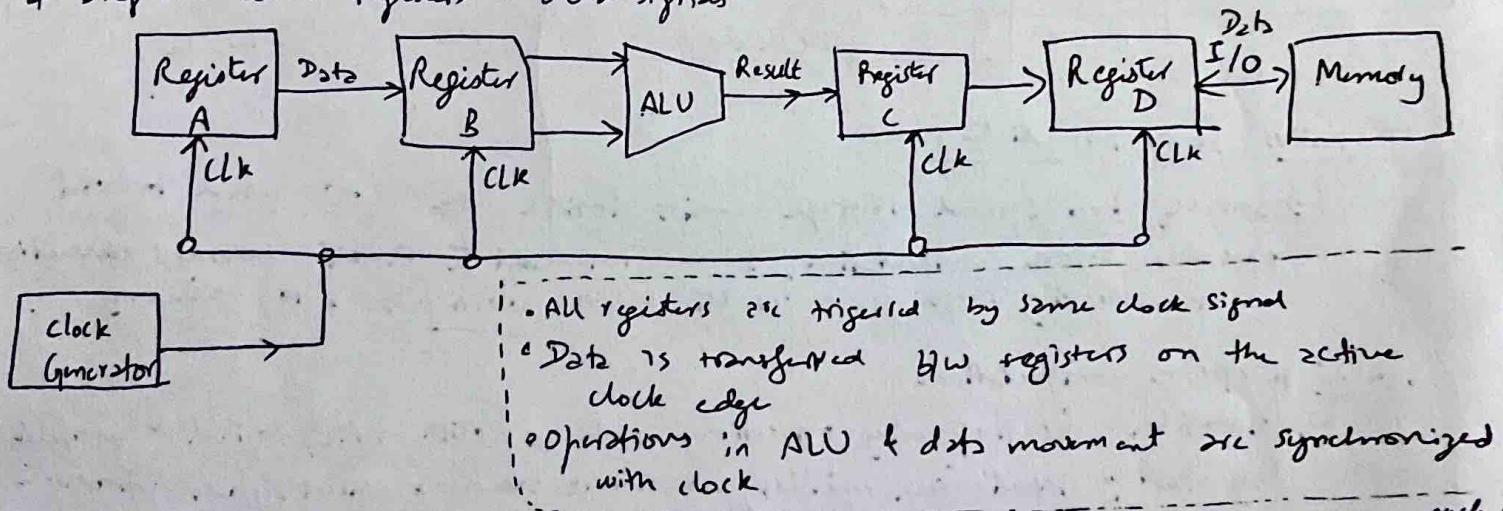
#### (1) Clocking Methodology

#### # Diagram: (Basic)

In the diagram, clock signal connects to all registers, meaning every register receives same timing signal, all operations inside CPU are coordinated using this single clock ensuring no component works independently, this is global synchronization. Data transfers happen at clock edges avoiding incorrect or unstable data transfer.



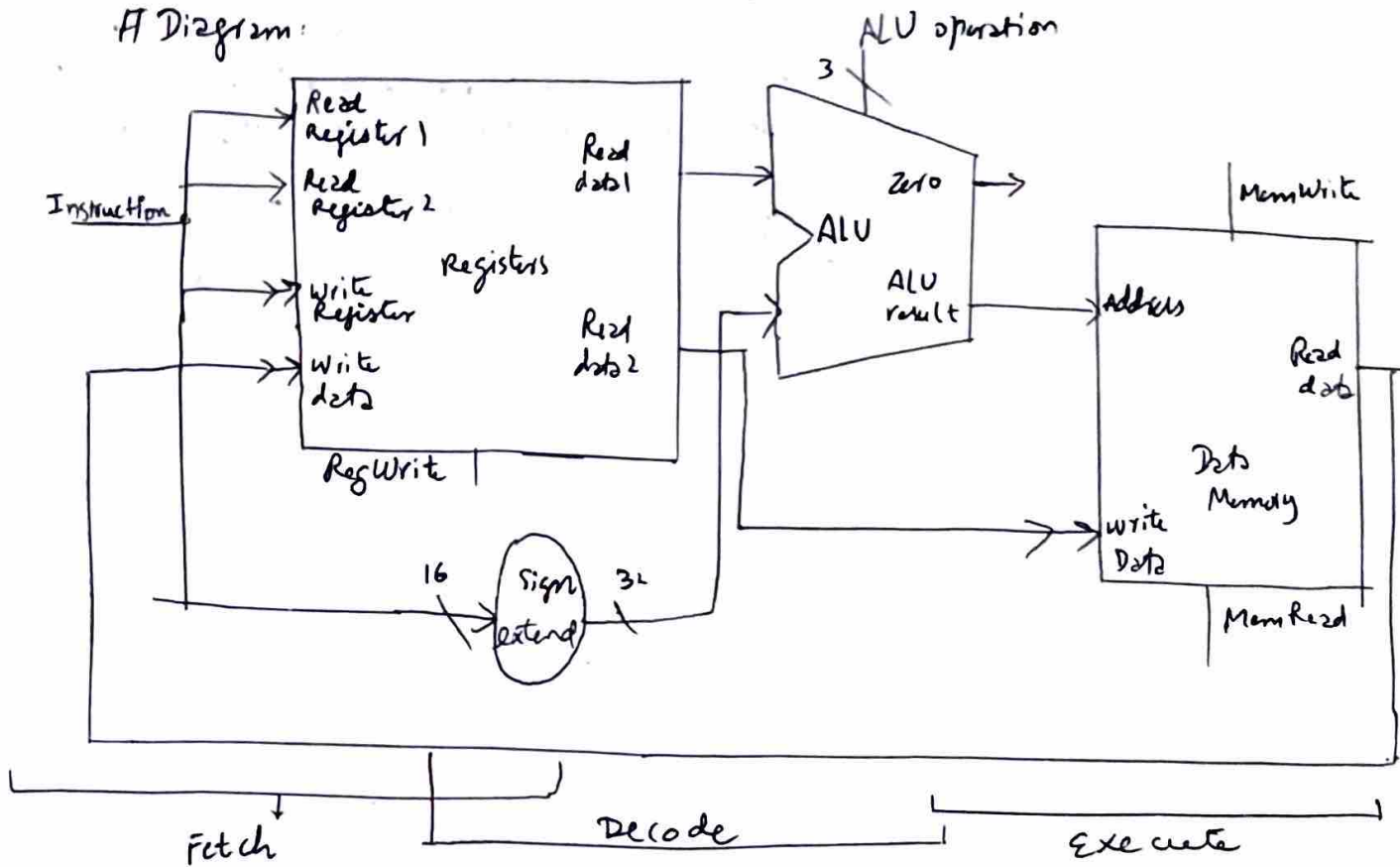
#### # Diagram with registers & clock signals:



\* Clocking methodology ensures that all operations in the processor occur in a synchronized & predictable manner, enabling efficient & error-free execution of instructions

## (11) Data Path of a simple Processor

AT Diagram:



• Registers store immediate values

↳ Small, high speed storage units inside the CPU, used to hold operands before computation, store immediate results during execution & temporarily keep final results before writing back to memory

• ALU performs computations

↳ responsible for performing operations such as arithmetic, logical & comparison by taking input from registers, processes it based on control signals & produces ~~output~~ results that are stored back in registers.

• Memory stores instructions & data

↳ used to store program instructions, input data & final results but during execution instructions are fetched from memory & data is read/written as required

• Buses transfer data b/w components

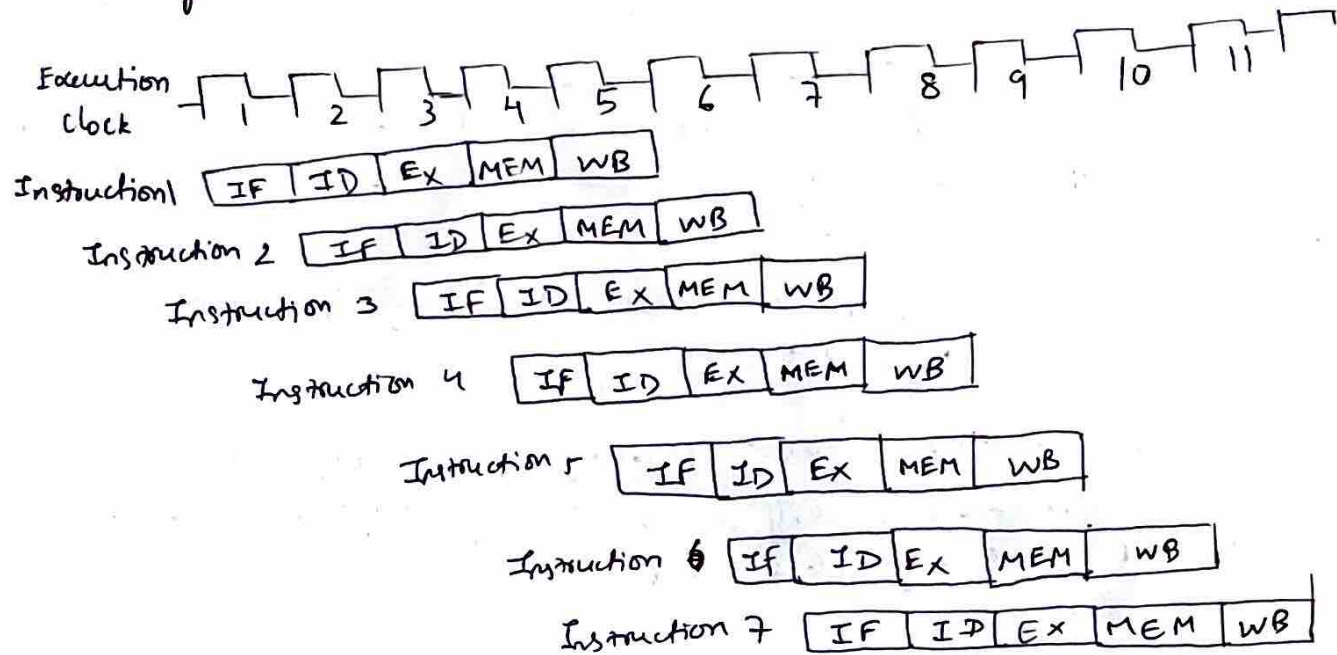
↳ communication pathways that connect different parts of CPU.

- ↳ Types: (i) Data Bus: transfer actual data
- (ii) Address Bus: carries memory addresses
- (iii) Control Bus: carries control signals

\* The datapath defines how data flows & is processed inside the CPU, enabling efficient execution of instructions through coordinated interaction b/w registers, ALU, memory & buses

### (III) Pipeline Stages

#### # Diagram



#### IF - Instruction Fetch

- ↳ Processor fetches instruction from memory using Program Counter (PC).
- ↳ The PC holds address of next instruction
- ↳ That instruction is loaded into IR (Instruction Register)
- ↳ The PC is then updated to point to the next instruction
- ↳ This stage ensures continuous supply of instructions to the pipeline

#### ID - Instruction Decode

- ↳ The fetched instruction is decoded to understand:
  - what operation needs to be performed
  - which registers are involved
- ↳ Actions Performed:
  - control unit generates control signals
  - Required registers are identified & their values are read
- ↳ This stage prepares instruction for execution

#### EX - Execute

- ↳ ALU performs arithmetic / logical operations
- ↳ Address calc is done for memory operations
- ↳ Branch conditions are evaluated
- ↳ Core processing stage of pipeline

#### (iv) MEM - Memory Access

- ↳ if instruction is a load/store, memory is accessed
  - load  $\rightarrow$  data is read from memory
  - store  $\rightarrow$  data is written to memory
- ↳ if not memory instruction, this stage is skipped logically
- ↳ connects CPU with main memory

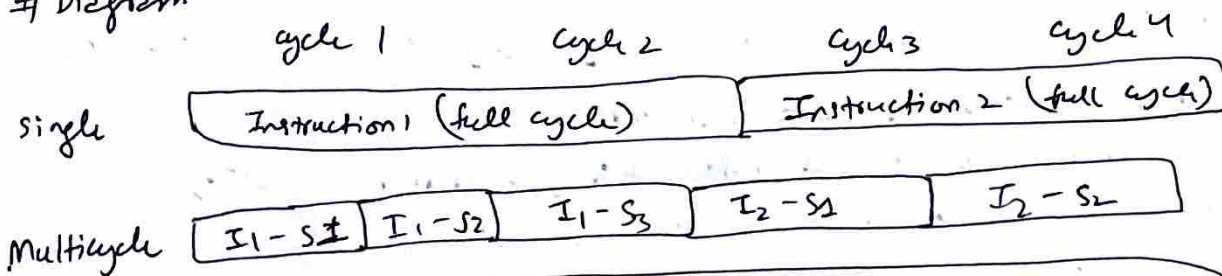
#### (v) WB - Write Back

- ↳ final stage where result from ALU or memory is written back into destination register

\* Instruction Pipelining improves processor efficiency by executing multiple instructions simultaneously in diff stages, reducing overall execution time without increasing clock speed

#### (iv) Single Cycle Datapath vs Multicycle Datapath

# Diagram



| feature            | Single cycle | Multicycle                   |
|--------------------|--------------|------------------------------|
| CPI                | 1 (fixed)    | variable per instruction     |
| Instruction steps  | Not divided  | Divided into arbitrary steps |
| Extra registers    | Not needed   | Required                     |
| clock cycle Timing | Long         | Short                        |

\* Single cycle is the simplest but wastes time on short instructions, whereas multicycle improves efficiency with short clock cycles but still processes one instruction at a time.

## (Section-1)

## (Can-Band Analytical Questions)

## (I) Data Dependency Issue:

↳ frequent dependencies cause data hazards, leading to incorrect execution or delays

Solution:

- forwarding reduces waiting time
- pipeline stalls ensure correctness

## (II) Amdahl's law calculation:

$$\text{Speedup} = \frac{1}{(1-0.8) + \frac{0.8}{4}} = \frac{1}{0.4} = 2.5$$

Indicating, performance is limited despite parallelization

## (III) Multi-cycle Efficiency:

↳ Multi-cycle processors:

- Use shorter clock cycles
- Avoid unnecessary delays
- Reuse components efficiently

Thus, they are more optimized than single-cycle processors

## (IV) Branch Hazard

↳ create control hazards, causing pipeline flushing.

Solution: • Branch prediction reduces misprediction

• Delayed balancing from registers & produces intermediate or final results

Spiral

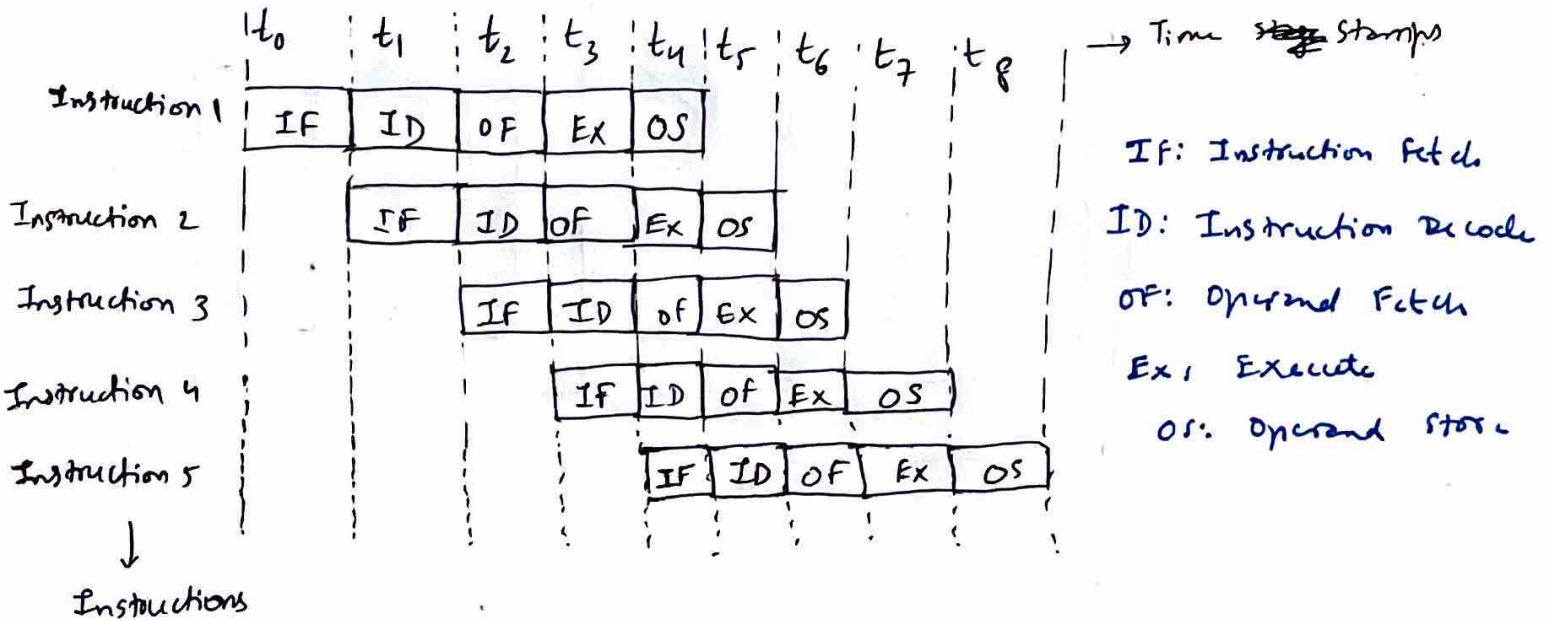
## (Section - D)

One Diagram + One Insight

(ODONI)

### (1) Pipeline

#Diagram:



• Stage explanation:

↳ EX stage: The Execute (EX) stage is responsible for planning arithmetic & logical operations using the ALU.

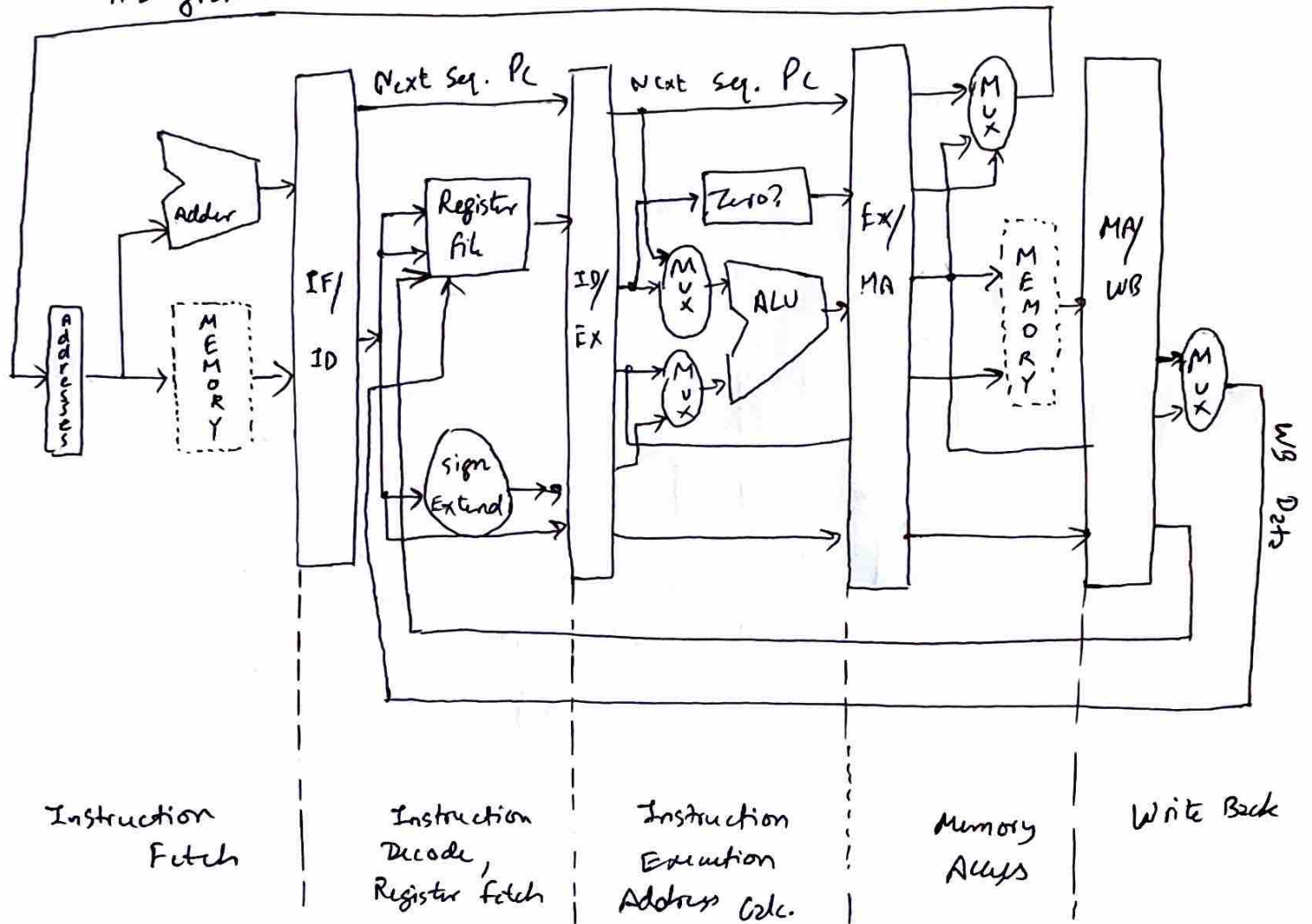
- It takes input from registers (decoded in ID stage)
- Performs operations like addition, subtraction, comparison
- Calculates memory addresses for LOAD/STORE instructions
- Evaluates branch conditions

• This stage is the core computation unit of the pipeline

\* The EX stage determines the outcome of operations, making it critical for both data processing & control flow decisions

## (11) Datapath of simple CPU

# Diagram:



# one design decision Explanation:

↳ MUX (Multiplexers):

Multiplexers are used to select b/w multiple input sources for the ALU or registers

- They allow flexibility in choosing operands
- Enable reuse of hardware components
- Reduce the need for multiple duplicated datapaths
- This improves efficiency & reduces hardware complexity

\* Using multiplexers makes the datapath flexible & cost-efficient by enabling controlled data selection

### (iii) Data Hazard

H Diagram:

|                        | $t_1$ | $t_2$ | $t_3$    | $t_4$    | $t_5$    | $t_6$    | $t_7$ | $t_8$ | $t_9$ |
|------------------------|-------|-------|----------|----------|----------|----------|-------|-------|-------|
| ADD $X_3, X_6, X_5$    | IF    | ID    | IE       | MEM      | RW $X_3$ | -        | -     | -     | -     |
| SUB $X_4, X_3, X_5$    | -     | IF    | ID $X_3$ | IE       | MEM      | RW       | -     | -     | -     |
| OR $X_6, X_3, X_7$     | -     | -     | IF       | ID $X_3$ | IE       | MEM      | RW    | -     | -     |
| AND $X_8, X_3, X_9$    | -     | -     | -        | IF       | ID $X_3$ | IE       | MEM   | RW    | -     |
| XOR $X_{12}, X_3, X_4$ | -     | -     | -        | -        | IF       | ID $X_3$ | IE    | MEM   | RW    |

ADD  $X_3, X_6, X_5$

- Result to be written in  $X_3$

$X_3$  is accessed in READ mode except the result of ADD to be available in  $X_3$

SUB  $X_4, X_3, X_5$   
OR  $X_6, X_3, X_7$   
AND  $X_8, X_3, X_9$   
XOR  $X_{12}, X_3, X_{10}$

-  $X_3$  has one of the operand  
-  $X_3$  has one of the operand  
-  $X_3$  has one of the operand  
-  $X_3$  has one of the operand

But the result of ADD written in  $X_3$  at  $t_5$

- when the execution of an instruction is dependent on the results of a prior instruction that's still being processed in pipeline, data hazards occur.

Now consider the above, the result of ADD instruction is written into the register  $X_3$  at  $t_5$  in the example. If the bubble isn't used to postpone the following SUB instruction, all the ~~three~~ three operations will use data from  $X_3$ , that is earlier in the ADD process. The program has gone haywire!

## \* Solutions to Data hazard:

### • Solution 1:

At the IF stage of the SUB instruction, add three bubbles - This will make it easier for SUB - ID to work at  $t_6$ . As a result, all subsequent instructions in the pipeline are similarly delayed.

### • Solution 2:

forwarding of Data - is the process of sending a result straight to that functional unit ~~which~~ which unit needs it. A result is transferred from one unit's output to another's input. The goal is to ~~ready~~ have solution ready for the next instruction as soon as possible.

### • Solution 3:

When generating executable code, the compiler can recognise data dependencies & reorganise (resequence) the instructions aptly. Making device easier to use.

### • Solution 4:

If the reordering is not possible, the compiler can detect & insert a no operation (NOP) instruction. NOP refers to a software-generated dummy instruction equivalent bubble. During the code optimization stage of the compilation process, the compiler examines data dependencies.